DC MOTOR SPEED CONTROL USING ARDUINO AND PWM

A PROJECT BASED LEARNING REPORT

Submitted by

Naresh A [210425105035]

Vishnu Varthan V G [210425105062]

This project report is submitted as part of the requirements for the completion of

EE5303 - CONTROL SYSTEM ENGINEERING



DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

CHENNAI INSTITUTE OF TECHNOLOGY

(An Autonomous Institution, Affiliated to Anna University, Chennai)

SARATHY NAGAR, KUNDRATHUR – 600 069

TAMIL NADU, INDIA

OCTOBER 2026

Vision of the Institution

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institution

IM1: To create next generation leaders by effective teaching learning methodologies and instil scientific spark in them to meet the global challenges.

IM2: To transform lives through deployment of emerging technology, novelty and sustainability.

IM3: To inculcate human values and ethical principles to cater to the societal needs.

IM4: To contribute towards the research ecosystem by providing a suitable, effective platform for interaction between industry, academia and R & D establishments.

IM5: To nurture incubation centres enabling structured entrepreneurship and start-ups.

Vision of the Department

To Excel in the emerging areas of Electrical and Electronics Engineering by imparting knowledge, relevant practices and inculcating human values to transform the students as potential resources to meet the industrial and societal needs

Mission of the Department

DM1: To provide strong fundamentals and technical skills through effective teaching learning Methodologies.

DM2: To transform lives of the students by fostering ethical values, creativity and novelty to become entrepreneurs and establish start-ups.

DM3: To habituate the students to focus on sustainable resources with optimal usage to ensure the welfare of the society.

DM4: To provide an ambience for research through collaborations with industry and academia.

DM5: To inculcate learning of emerging technologies for pursuing higher studies leading to lifelong learning.

CHENNAI INSTITUTE OF TECHNOLOGY

Transforming Lives

CHENNAI INSTITUTE OF TECHNOLOGY

(Approved by AICTE, New Delhi & Affiliated to Anna University Chennai), Sarathy Nagar, Kundrathur, Chennai - 600069.

BONAFIDE CERTIFICATE

Certified that this project report entitled **"DC MOTOR SPEED CONTROL USING ARDUINO AND PWM"** is the Bonafide work of **Naresh A [210425105035], Vishnu Varthan V G[210425105062]** who carried out the EE5303 Control System Engineering - Project Based Learning under my supervision during the academic year **2026-2027**.

SIGNATURE

Dr. N. MALATHY, M.E. Ph.D.,

HEAD OF THE DEPARTMENT

Assistant Professor,

Department of Electrical and Electronics Engineering,

Chennai Institute of Technology,

Chennai – 600069.

SIGNATURE

Mr.Ezhilmaran M, M.E.,

SUPERVISOR

Assistant Professor,

Department of Electrical and Electronics Engineering,

Chennai Institute of Technology,

Chennai-600069.

Submitted for Project Viva Voce Examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDMENT

It is our first and foremost duty to express our heartily gratitude and allegiance to our beloved chairman **Sri. P. SRIRAM** for providing us a good working environment, facilities and opportunity to carry out the research work with full freedom in and out of the campus.

We would like to express our deepest gratitude to our Principal **Dr. A. RAMESH**, Ph.D., for being our inspiration throughout the entire period of course.

We greatly appreciate and convey our heartfelt thanks to **Dr. N. MALATHY ,M.E, Ph.D.,** Assistant Professor & Head – Electrical and Electronics Engineering, for her continuous support and valuable suggestions throughout the duration of our core course project.

We express our gratefulness to **Dr. J. POORNIMA, M.E, Ph.D.,** Assistant Professor and Program Coordinator, Department of Electrical and Electronics Engineering, Chennai Institute of Technology for her constructive criticism in the reviews.

It gives us an immense pleasure to express our deep sense of gratitude to our project guide, **Mr.Ezhilmaran M,M.E.** Assistant Professor, Faculty of Electrical and Electronics Engineering for her valuable guidance, constant inspiration, encouragement, and motivation.

We express our gratitude and sincere thanks to all the faculty and supporting staff of **Department of Electrical and Electronics Engineering** for their valuable suggestions and support during the course of our project work.

Naresh A [210425105035]
Vishnu Varthan VG [210425105062]

ABSTRACT

Speed control of direct-current (DC) motors is needed in robotics, fans, pumps, conveyors and many other machines. Conventional methods based on series resistors or linear regulators waste power as heat and give poor control. This project presents a simple and low-cost DC motor speed controller that uses Pulse Width Modulation (PWM) with an Arduino UNO. A 10 kΩ potentiometer connected to the analog input A0 acts as the speed-setting device. The 10-bit analog-to-digital converter of the ATmega328P converts the wiper voltage into a number between 0 and 1023, which the program scales to a PWM value between 0 and 255 with the map() function and writes to pin 9 with analogWrite(). Pin 9 is connected to the ENA input of an L298N dual H-bridge driver, so the driver switches the supply of an external power source to a DC geared motor at the selected duty cycle. The average voltage across the motor, and therefore its speed, rises in proportion to the duty cycle, from standstill at 0 % to full speed at 100 %. The direction of rotation is fixed by the logic levels of the IN1 and IN2 pins, and the potentiometer and PWM values are displayed on the serial monitor for testing. The report presents the circuit, the program, the working principle, the relation between the knob position, the duty cycle and the motor voltage, and the practical points that make the circuit work, such as removing the ENA jumper and joining the grounds. The system is efficient and easy to build, and it can be extended with encoder feedback, bidirectional control and a more efficient driver.

Keywords: DC motor, speed control, PWM, Arduino UNO, L298N, H-bridge, potentiometer

TABLE OF CONTENTS

CHAPTER No.	TITLE	PAGE No.
	ABSTRACT	**vi**
	LIST OF FIGURES	**ix**
	LIST OF TABLES	**x**
	LIST OF ABBREVIATIONS	**xi**
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	4
	1.3 OBJECTIVES	5
	1.4 SCOPE OF THE PROJECT	6
	1.5 PROPOSED SOLUTION	7
2	**LITERATURE SURVEY**	**8**
	2.1 EXISTING SYSTEMS	8
	2.2 LIMITATIONS OF EXISTING SYSTEMS	9
	2.3 COMPARISON OF EXISTING APPROACHES	10
	2.4 RESEARCH GAP	11
3	**SYSTEM REQUIREMENTS**	**12**
	3.1 HARDWARE REQUIREMENTS	12
	3.2 SOFTWARE REQUIREMENTS	16
4	**SYSTEM DESIGN**	**18**
	4.1 BLOCK DIAGRAM	18
	4.2 CIRCUIT DIAGRAM	20
	4.3 FLOWCHART	23
5	**IMPLEMENTATION**	**26**
	5.1 HARDWARE IMPLEMENTATION	26
	5.2 SOFTWARE IMPLEMENTATION	28
6	**WORKING OF THE SYSTEM**	**33**
7	**RESULTS AND DISCUSSION**	**38**

	7.1 EXPERIMENTAL SETUP	38
	7.2 OBSERVED RESULTS	38
	7.3 DISCUSSION	42
8	**ADVANTAGES, LIMITATIONS AND APPLICATIONS**	**46**
	8.1 ADVANTAGES	46
	8.2 LIMITATIONS	46
	8.3 APPLICATIONS	47
9	**CONCLUSION AND FUTURE SCOPE**	**49**
	9.1 CONCLUSION	49
	9.2 FUTURE SCOPE	50
	REFERENCES	**51**

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
1.1	PWM signals at different duty cycles and their average voltage	3
3.1	Operation of an H-bridge for forward and reverse rotation	14
3.2	Layout of the L298N motor driver module	14
4.1	Block diagram of the proposed system	18
4.2	Circuit diagram of the proposed system	21
4.3	Flowchart of the system operation	24
5.1	Mapping of the potentiometer reading to the PWM value	29
6.1	Signal chain from the knob position to the motor voltage	36
7.1	PWM duty cycle versus potentiometer reading	40
7.2	Calculated average motor voltage versus duty cycle	40
7.3	Calculated average motor voltage for different supply voltages	42

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
2.1	Comparison of existing approaches	10
3.1	Hardware requirements	12
3.2	Software requirements	16
4.1	Pin connections of the Arduino with the modules	21
4.2	L298N module terminals used in the project	22
5.1	Mapping of potentiometer readings to PWM values	29
5.2	Common problems and remedies	32
6.1	Logic states of the L298N for one channel	34
6.2	Signal values at each stage for three knob settings	36
7.1	PWM values, duty cycle and motor voltage for different knob positions	39
7.2	Functional tests of the speed controller	41
7.3	Calculated average motor voltage for different supply voltages	41

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ADC	Analog-to-Digital Converter
AVR	Alf and Vegard's RISC (Atmel microcontroller family)
DC	Direct Current
EMF	Electromotive Force
ENA	Enable A (L298N input)
GND	Ground
H-bridge	Four-switch circuit for driving a load in both directions
I/O	Input / Output
IC	Integrated Circuit
IDE	Integrated Development Environment
LCD	Liquid Crystal Display
MCU	Microcontroller Unit
MOSFET	Metal Oxide Semiconductor Field Effect Transistor
PBL	Project-Based Learning
PID	Proportional–Integral–Derivative
PWM	Pulse Width Modulation
RPM	Revolutions Per Minute
SRAM	Static Random Access Memory
TTL	Transistor–Transistor Logic
USB	Universal Serial Bus
VCC	Positive Supply Voltage

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Electric motors convert electrical energy into mechanical energy and sit at the heart of almost every machine that moves, from household fans and washing machines to conveyor belts, robots and electric vehicles. Among the different types of motors, the direct-current (DC) motor remains one of the most widely used in small and medium power applications. Its popularity comes from a simple construction, a high starting torque, a quick response and, above all, the ease with which its speed can be varied over a wide range. Because the speed of a DC motor depends directly on the voltage applied to its armature, controlling that voltage is enough to control the speed.

In many applications the speed of the motor cannot be fixed. A robotic vehicle has to slow down while turning, a ventilation fan has to run faster when the room heats up, and a small pump has to deliver a different flow rate at different times. For this reason a practical motor drive must allow the user, or the control program, to change the speed smoothly while the machine is running.

1. Methods of Varying the Voltage Across a DC Motor

The simplest way to reduce the voltage across a motor is to connect a series resistor or a rheostat. This method is easy to understand, but it wastes a large part of the electrical power as heat in the resistor, and the speed changes sharply whenever the load changes. A linear voltage regulator behaves in a similar way: the part of the supply voltage that is not delivered to the motor is dissipated as heat in the regulator. Both approaches are therefore inefficient and become impractical as the motor current increases.

A far better solution is to switch the supply to the motor rapidly on and off and to let the motor respond to the average value of the resulting voltage. This technique is called Pulse Width Modulation (PWM). Since the switch is either

fully on (very small voltage across it) or fully off (no current through it), the power lost in the switching element is small, and the method stays efficient even at low speeds.

2. Pulse Width Modulation

A PWM signal is a rectangular wave of fixed frequency whose on-time is varied. The fraction of one period during which the signal is high is called the duty cycle, D. If the on-time is T_{on} and the off-time is T_{off}, then the period is $T = T_{on} + T_{off}$ and the duty cycle is

$$D = T_{on} / (T_{on} + T_{off}) = T_{on} / T \tag{1.1}$$

When a supply of voltage V_s is switched with this duty cycle, the average voltage seen by a load that cannot follow the fast switching is

$$V_{avg} = D \times V_s \tag{1.2}$$

Figure 1.1 shows a 5 V PWM signal at duty cycles of 25 %, 50 %, 75 % and 100 %. The average voltage rises in direct proportion to the duty cycle. A motor winding has inductance and the rotor has inertia, so both smooth the pulses, and the motor behaves as if it were supplied with a steady voltage equal to V_{avg}. At a duty cycle of 0 % the average voltage is zero and the motor stops, and at 100 % the motor receives the full supply voltage.

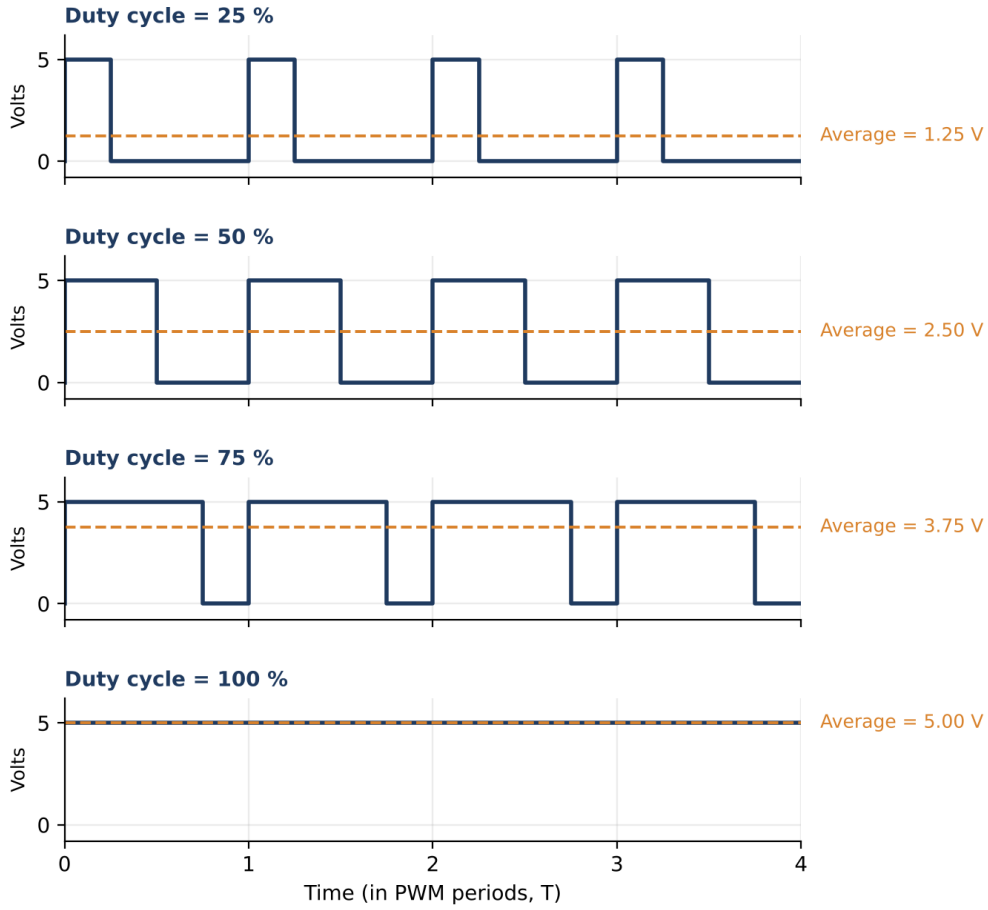


Figure 1.1 PWM signals at different duty cycles and their average voltage

3. Role of the Microcontroller

Generating a PWM signal in hardware, and changing its duty cycle on demand, is a natural job for a microcontroller. The Arduino UNO, which is based on the ATmega328P microcontroller, is an open-source development board that provides built-in PWM outputs, a 10-bit analog-to-digital converter and a simple programming environment. Its low cost, large user community and ease of use have made it one of the most popular platforms for student projects, prototyping and small embedded products.

A microcontroller pin, however, can supply only about 20 mA, while even a small geared motor draws hundreds of milliamperes, and the winding of a motor produces voltage spikes when the current is interrupted. A power interface is therefore needed between the Arduino and the motor. In this project the L298N

dual H-bridge driver module plays that role. It takes the low-power PWM and direction signals from the Arduino and switches the much larger motor current from a separate power supply.

4. Why PWM Control with the Arduino and the L298N?

Several alternatives exist for driving a small DC motor. A ready-made speed-controller board is convenient but hides the working principle and offers no scope for modification. A discrete MOSFET bridge is efficient but needs gate-drive circuits and careful design. A linear regulator is easy to use but wasteful. The combination of an Arduino and an L298N module sits between these extremes: both parts are cheap and available everywhere, the module already contains the bridge, the flyback diodes and the direction logic, and the Arduino program is only a few lines long. This makes the combination ideal for a project-based learning exercise in which every stage of the signal path can be examined and measured.

5. Importance of Speed Control in Practice

The ability to change the speed of a motor has practical value far beyond convenience. Running a fan or a pump slower when the full output is not needed saves electrical energy. Starting a motor at a low speed and raising it gradually reduces the mechanical shock on gears and belts and extends the life of the machine. In robots and vehicles, speed control makes smooth manoeuvring, line following and precise positioning possible. In process equipment it allows the output to be matched to the demand. A PWM-based controller offers all of these benefits at a very low cost.

1.2 PROBLEM STATEMENT

Small motor-driven systems built by students and hobbyists, as well as many low-cost industrial products, still suffer from three recurring problems in the way their speed is controlled:

- **Inefficient speed control:** Resistor- and regulator-based methods waste power as heat, give poor speed regulation and make the system bulky and unreliable when the motor current is high.
- **No simple adjustable interface:** Many available circuits have a fixed speed or require complicated analog circuitry to change it. A user needs an easy, intuitive way, such as turning a knob, to vary the speed continuously from zero to maximum.
- **Unsafe direct connection:** A motor cannot be connected directly to a microcontroller pin. The limited pin current and the inductive voltage spikes can permanently damage the microcontroller, so a proper driver stage and a correct grounding arrangement are essential.

A low-cost and well-documented solution is therefore required that controls the speed of a DC geared motor efficiently with PWM, allows the speed to be changed smoothly by hand, and protects the controller from the motor.

If these problems are not addressed, the result is a system that overheats, wastes energy, behaves unpredictably when the load changes, and may be damaged by a wiring mistake. For learners, an undocumented circuit that does not work at the first attempt, for example because the enable jumper of the driver was left in place, is also a serious obstacle. The project therefore pays special attention to explaining and recording the practical details of the circuit.

1.3 OBJECTIVES

To solve the problems described above, the DC Motor Speed Control Using Arduino and PWM project sets the following engineering objectives:

- **Objective 1: Manual Speed Command:** To read the position of a potentiometer through the analog input A0 of the Arduino and to convert the 10-bit reading (0 to 1023) into a speed command.

- **Objective 2: PWM Generation:** To scale the speed command to an 8-bit value (0 to 255) and to generate a PWM signal of the corresponding duty cycle on pin 9 of the Arduino using the analogWrite() function.
- **Objective 3: Safe Power Interface:** To drive a DC geared motor from an external power supply through the L298N motor driver, with the PWM signal applied to the ENA pin and a common ground between the Arduino, the driver and the supply.
- **Objective 4: Verification and Documentation:** To display the potentiometer value and the PWM value on the serial monitor, to study how the motor speed varies with the duty cycle, and to document the circuit, program and results clearly enough for the project to be repeated.

1.4 SCOPE OF THE PROJECT

The project covers the design, construction and testing of a prototype speed controller for a DC geared motor. The scope includes reading a potentiometer with the Arduino ADC, mapping the reading to a PWM value, driving the L298N enable pin with the PWM signal, fixing the direction of rotation in software, and observing the behaviour of the motor over the full range of the potentiometer.

The system is an open-loop speed controller: it sets the average motor voltage but does not measure the actual speed. The prototype is built on a breadboard with a low-voltage supply for laboratory demonstration. Closed-loop speed regulation with an encoder, bidirectional control from the user interface and high-power drives are outside the scope of this work, although they are discussed as future enhancements in Chapter 9.

Inclusions and Exclusions

- **Included:** potentiometer input, 10-bit ADC reading, PWM generation on pin 9, L298N interfacing, fixed-direction operation, serial monitoring, calculation and plotting of the duty cycle and the average motor voltage.

- **Excluded:** speed measurement and feedback, bidirectional control from the user interface, high-power motors, thermal design of the driver for continuous heavy loads and the design of a printed circuit board.

1.5 PROPOSED SOLUTION

The proposed system uses an Arduino UNO (or Nano) as the control unit, a potentiometer as the speed-setting device and an L298N module as the motor driver. The solution works in three stages:

- **Input acquisition:** The potentiometer is connected as a voltage divider between 5 V and ground, and its wiper is connected to the analog pin A0. Rotating the knob changes the voltage at A0 from 0 V to 5 V, and the 10-bit ADC converts it into a number between 0 and 1023.
- **PWM generation:** The program uses the map() function to convert the number 0–1023 into a PWM value of 0–255 and writes it to pin 9 with analogWrite(). The hardware timer of the ATmega328P then produces a square wave on pin 9 whose duty cycle equals the PWM value divided by 255.
- **Power amplification:** Pin 9 is connected to the ENA pin of the L298N. The driver switches the motor supply on and off at the PWM rate, so the average voltage across the motor, and therefore its speed, follows the knob position. Two further pins, IN1 and IN2, fix the direction of rotation.

Organisation of the Report

Chapter 2 reviews the existing work on PWM-based DC motor speed control. Chapter 3 lists the hardware and software requirements, and Chapter 4 presents the system design with the block diagram, circuit diagram and flowchart. Chapter 5 describes the hardware and software implementation, and Chapter 6 explains the working of the system step by step. Chapter 7 discusses the results, Chapter 8 lists the advantages, limitations and applications, and Chapter 9 gives the conclusion and the future scope.

CHAPTER 2
LITERATURE SURVEY

2.1 EXISTING SYSTEMS

Speed control of DC motors by pulse width modulation has been studied widely, and the published work ranges from simulation studies to complete microcontroller-based drives. This section reviews representative papers that are closely related to the present project.

Joshi and Thakare [1] presented a microcontroller-based approach in which the power to a small DC motor is switched on and off by MOSFETs arranged as an H-bridge. They showed that accurate and precise control of small DC motors is possible without complicated circuitry or costly components, and they varied the speed by changing the duty cycle from 0 to 90 percent. Islam and Tripathi [2] studied DC motor speed control through pulse width modulation and implemented the idea in a MATLAB simulation, which allows the effect of the duty cycle on the motor to be examined before any hardware is built.

Several authors have used the Arduino platform. Mane et al. [3] developed an Arduino-based speed control system in which an L298N H-bridge drives the motor, a Hall-effect current sensor and an LM35 temperature sensor monitor the operating condition, and the parameters are displayed and controlled through LabVIEW. They explain that varying the pulse width varies the average voltage across the motor and hence its speed. Banerjee et al. [4] built a controller from an Arduino UNO R3, an L298N driver, a 100 kΩ potentiometer and an IR sensor. The speed can be set manually with the potentiometer or by changing the PWM value in the program, and the IR sensor works as a tachometer whose RPM reading is shown on an LCD.

At the power-electronics level, Özer et al. [5] designed an H-bridge driver for a dsPIC30F4011 microcontroller using NPN and PNP MOSFETs, with soft switching to protect the bridge during direction changes. The circuit was tested

on a 300 W DC motor at duty cycles from 30 % to 100 % and included current sensing for over-current protection. The behaviour of the motor driver itself, such as the saturation voltage, the logic truth table and the need for flyback diodes, is documented in the L298 datasheet [6], while the PWM hardware of the ATmega328P is described in its datasheet [7] and the analogWrite() function in the Arduino reference [8]. Standard textbooks such as Rashid [9] and Chapman [10] provide the theory of DC motors and of switch-mode power conversion.

Taken together, these works establish three points that are relevant to the present project. First, PWM is an accepted and effective way of varying the average voltage across a DC motor, and the duty cycle gives a direct handle on the speed. Second, an H-bridge, whether built from MOSFETs or provided by an integrated driver such as the L298, is the standard way of switching the motor current and of selecting its direction. Third, the Arduino platform has already been shown to work as the controller in such systems, either alone or together with sensors and a PC interface.

2.2 LIMITATIONS OF EXISTING SYSTEMS

Among the works reviewed here, some studies are limited to simulation and do not show how a real motor responds to the PWM signal, including effects such as friction and the voltage drop of the driver. Others use heavier hardware, such as a dsPIC controller with a discrete MOSFET bridge or a computer running LabVIEW, which is more than a student or a small product needs and raises the cost and the effort required to reproduce the system.

The Arduino-based designs add extra sensors, a display or a PC interface. These additions are useful, but they hide the basic relationship between the knob position, the PWM value and the motor behaviour, which is what a beginner needs to understand first. Practical details that often cause failures, for example removing the ENA jumper of the L298N module and joining the grounds, are seldom discussed clearly. Finally, the papers reviewed here do not tabulate the

potentiometer reading, PWM value, duty cycle and expected motor voltage side by side.

Observations from the Review

- **Simulation versus hardware:** Simulation is useful for understanding the effect of the duty cycle, but only a hardware prototype shows the dead zone, the driver voltage drop and the heating.
- **Complexity versus clarity:** Added sensors, displays and PC interfaces make a project more capable but also harder for a beginner to follow and to reproduce.
- **Driver choice:** The L298N is the most common driver in Arduino-based work because of its availability and low cost, in spite of its voltage drop.
- **Documentation:** Practical wiring details deserve as much attention as the theory, because most failures of a first prototype come from them.

2.3 COMPARISON OF EXISTING APPROACHES

Table 2.1 compares the reviewed works with the proposed system. “Not stated” means that the source does not mention the feature in the portions that were read.

Table 2.1 Comparison of existing approaches

Work	Controller	Driver	Speed setting / feedback
Joshi and Thakare [1]	Microcontroller	MOSFET H-bridge	Duty cycle varied from 0 to 90 %; feedback not stated
Islam and Tripathi [2]	MATLAB simulation	Simulated PWM drive	Duty cycle in simulation; no hardware
Mane et al. [3]	Arduino with LabVIEW	L298N	PWM from PC interface; Hall current sensor and LM35 monitoring
Banerjee et al. [4]	Arduino UNO R3	L298N	Potentiometer or program value; IR tachometer with LCD
Özer et al. [5]	dsPIC30F4011	MOSFET H-bridge	Duty cycles of 30 % to 100 %; current sensing
Proposed system	Arduino UNO / Nano	L298N	Potentiometer on A0, PWM on pin 9; open loop, serial monitor display

2.4 RESEARCH GAP

The literature shows that PWM is a proven and efficient method for controlling DC motors and that the Arduino and L298N combination is common. However, the available work either concentrates on advanced features such as feedback and PC-based supervision or stays at the level of simulation. A compact, low-cost and clearly documented implementation that links the potentiometer reading, the PWM value, the duty cycle and the average motor voltage, and that records the practical points needed to make it work, is still useful, especially for teaching.

The present project fills this gap. It builds a minimum-component, open-loop speed controller around the Arduino UNO, the L298N and a potentiometer, explains every stage of the signal path, and documents the circuit, the program and the observations in a form that can be reproduced and later extended with an encoder, a display or closed-loop control.

CHAPTER 3
SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

This section lists the physical components required to build the speed controller. Table 3.1 gives each component with its model or rating and its function in the system, and the main components are described in more detail below.

Table 3.1 Hardware requirements

S.No	Component	Model / Rating	Function
1	Microcontroller board	Arduino UNO (ATmega328P) or Arduino Nano	Reads the potentiometer, generates the PWM signal and the direction signals
2	Motor driver	L298N dual H-bridge module	Switches the motor current from the external supply according to the Arduino signals
3	DC motor	DC geared motor (voltage rating to match the supply)	Load whose speed is controlled
4	Potentiometer	10 kΩ linear (rotary)	Sets the required speed as a variable voltage
5	Power supply	External DC supply suitable for the motor (for example 6–12 V)	Provides the motor power through the L298N
6	Jumper wires	Male-to-male and male-to-female	Interconnection of the modules
7	Breadboard (optional)	Standard solderless board	Mounting the potentiometer and wiring the prototype
8	USB cable	USB Type-A to Type-B (UNO) or Mini/Micro-B (Nano)	Programming and 5 V logic supply from a computer

1. Arduino UNO / Nano (ATmega328P)

The Arduino UNO is built around the 8-bit ATmega328P microcontroller that runs at 16 MHz. It has 14 digital input/output pins, six of which (pins 3, 5, 6, 9, 10 and 11) can produce 8-bit PWM, and six analog input pins connected to a

10-bit ADC. The board operates at 5 V logic, and the microcontroller provides 32 KB of flash memory for the program and 2 KB of SRAM. The Arduino Nano uses the same microcontroller in a smaller package, so the same circuit and program can be used on either board. In this project pin A0 reads the potentiometer, pin 9 supplies the PWM signal, and pins 8 and 7 set the direction.

If an Arduino Nano is used, the pin numbers of the circuit remain unchanged: the potentiometer is connected to A0, the enable input to D9 and the direction inputs to D8 and D7, because the Nano exposes the same pins of the same microcontroller. Only the connector and the programming cable differ. Pin D9 is also a PWM-capable pin on the Nano, so the program of Chapter 5 runs on either board without any modification.

2. L298N Motor Driver Module

The L298N is a dual full-bridge driver integrated circuit that can drive two DC motors, or one stepper motor, with a supply of up to 46 V and a current of up to 2 A per channel [6]. Each channel contains an H-bridge made of four switching transistors, as shown in Figure 3.1. Closing the switches S1 and S4 passes the current through the motor in one direction, and closing S2 and S3 reverses it. A separate enable input switches the whole bridge on and off, and this is the input to which the PWM signal is applied.

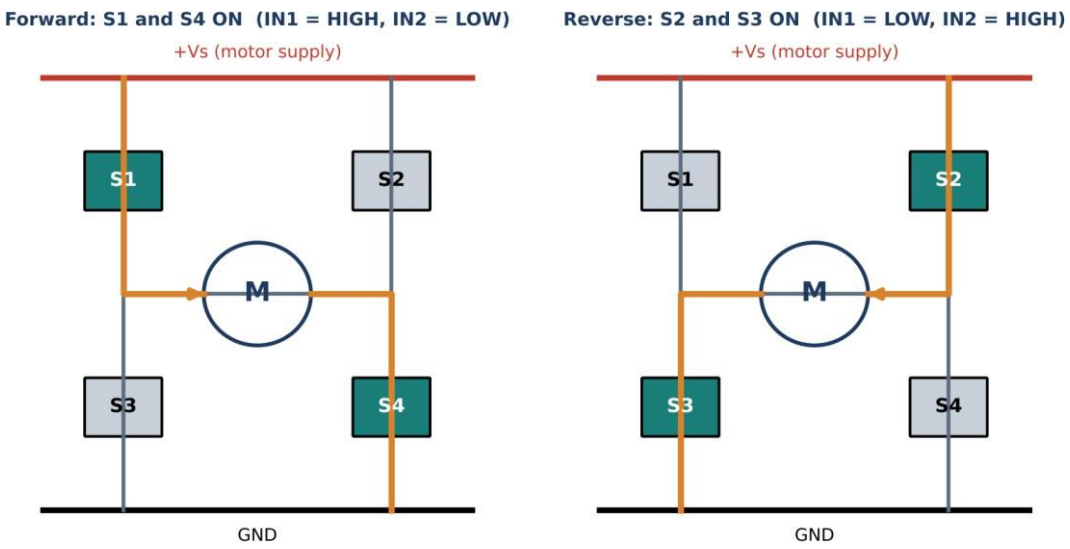


Figure 3.1 Operation of an H-bridge for forward and reverse rotation

The commonly available L298N breakout module adds screw terminals, flyback diodes and an on-board 5 V regulator to the integrated circuit. The terminals of the module that are used in this project are listed in Table 4.2 in Chapter 4.

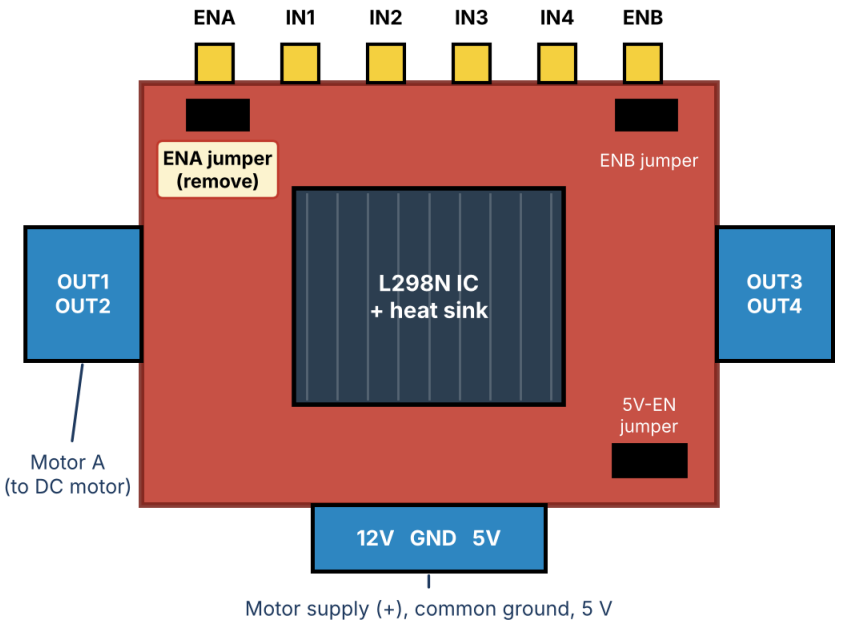


Figure 3.2 Layout of the L298N motor driver module

Figure 3.2 shows the typical layout of the module. The two blue terminals at the left and right carry the outputs of the two channels (OUT1–OUT2 for motor

A and OUT3–OUT4 for motor B). The terminal at the bottom takes the motor supply (12V), the ground and the 5 V line. The row of header pins at the top carries the control inputs: ENA, IN1 and IN2 for channel A, and IN3, IN4 and ENB for channel B. Only channel A is used in this project, and the ENA jumper next to the header is removed so that the PWM signal can control the enable input.

3. DC Geared Motor

A geared DC motor combines a small DC motor with a gear train that reduces the speed and multiplies the torque at the output shaft. This makes it suitable for robots, small conveyors and similar loads that need a low speed and a reasonable torque. The speed of the motor at no load is roughly proportional to the voltage applied to it, and the gear box reduces this speed by the gear ratio. The motor chosen for the prototype must be rated for the voltage of the external supply and must not draw more than the current limit of the L298N channel.

4. Potentiometer

A potentiometer is a three-terminal variable resistor. When its outer terminals are connected across 5 V and ground, the wiper terminal gives a voltage between 0 V and 5 V that depends linearly on the shaft angle. A 10 kΩ linear potentiometer is used because it loads the 5 V supply by only 0.5 mA and its output impedance is low enough for the Arduino ADC to read accurately.

5. Power Supply, Wires and Breadboard

The motor is not powered from the Arduino. A separate DC supply with a voltage and current rating suitable for the motor is connected to the 12V terminal of the L298N, and its negative terminal is joined to the grounds of the driver and the Arduino. The Arduino itself is powered from the USB cable or from a 7–12 V adapter. Jumper wires and a breadboard are used to connect the potentiometer and the control signals so that the prototype can be modified easily.

6. Selection Criteria for the Components

- **Motor voltage rating:** The external supply should be chosen so that, after the voltage drop of the L298N, the motor receives approximately its rated voltage at 100 % duty cycle. A supply much higher than the rating makes it necessary to limit the maximum PWM value in the program.
- **Motor current:** The running current of the motor must be well below the 2 A limit of one channel of the L298N, and the stall current should also be checked, since a jammed motor draws the largest current.
- **Supply current capacity:** The power supply must be able to deliver the stall current of the motor without a large fall in voltage.
- **Potentiometer value:** A value between 5 kΩ and 10 kΩ gives a good balance between a small current drain from the 5 V supply and a low source impedance for the ADC.

3.2 SOFTWARE REQUIREMENTS

This section outlines the development environment and the language features used to program the Arduino. Table 3.2 lists the software items and their purpose. No external library is required, because every function used is part of the standard Arduino core.

Table 3.2 Software requirements

S.No	Item	Name / Version	Purpose
1	Development environment	Arduino IDE v2.x	Writing, compiling and uploading the program to the board
2	Programming language	C/C++ (Arduino language)	Control logic of the speed controller
3	Function	pinMode(), digitalWrite()	Configures the pins as outputs and sets the direction pins IN1 and IN2
4	Function	analogRead()	Reads the potentiometer voltage as a 10-bit number (0–1023)
5	Function	map()	Scales the 0–1023 reading to the 0–255 PWM range
6	Function	analogWrite()	Generates the PWM signal on pin 9

S.No	Item	Name / Version	Purpose
7	Tool	Serial Monitor and Serial Plotter (9600 baud)	Displays the potentiometer value and the PWM value for testing

The function analogRead() starts a conversion of the ADC and returns an integer between 0 and 1023. Each count corresponds to 5 V / 1024, or about 4.88 mV, and a conversion takes about 100 µs on the UNO. The function map(value, fromLow, fromHigh, toLow, toHigh) rescales a number linearly using integer arithmetic. The function analogWrite(pin, value) accepts a value from 0 (always low) to 255 (always high) and sets the duty cycle to value/255. On the pins used here, pin 9 outputs a PWM signal of approximately 490 Hz [8], which lies in the audible range but is fast enough for the motor to see only the average voltage.

Setting Up the Software

- Install the Arduino IDE and connect the board to the computer with the USB cable.
- Select the correct board (Arduino UNO or Arduino Nano) and the serial port from the Tools menu. A Nano clone may need a USB-to-serial driver to be installed first.
- Type or paste the program of Section 5.2 and press Verify to check that it compiles without errors.
- Press Upload to transfer the program to the board, and open the Serial Monitor at 9600 baud to view the potentiometer and PWM values.

The Serial Plotter of the IDE can also be used to watch the two values as a graph while the knob is turned, which is a convenient way of checking the linear relation between them. For this purpose the second program of Chapter 5 prints the two values separated by a comma.

CHAPTER 4
SYSTEM DESIGN

4.1 BLOCK DIAGRAM

Figure 4.1 shows the overall block diagram of the DC motor speed controller. The design is divided into four functional parts: the input, the controller, the driver and the load, supported by a power supply.

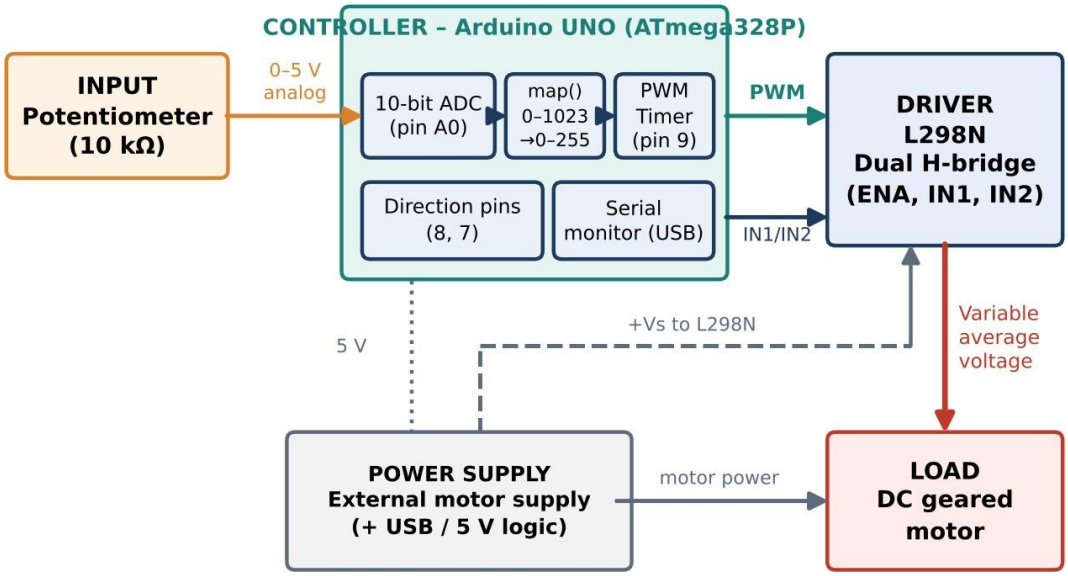


Figure 4.1 Block diagram of the proposed system

The input block is a 10 kΩ potentiometer connected between 5 V and ground. Its wiper gives an analog voltage between 0 V and 5 V that represents the required speed. The controller block is the Arduino UNO. Inside the ATmega328P the voltage is digitised by the 10-bit ADC on pin A0, the program scales the result with the map() function, and a hardware timer outputs the PWM signal on pin 9. Two further pins, 8 and 7, are set to fixed logic levels to select the direction of rotation, and the values are also sent to the serial monitor through the USB port.

The driver block is the L298N dual H-bridge. It receives the PWM signal at its ENA input and the direction signals at IN1 and IN2, and it switches the

motor supply to the motor terminals OUT1 and OUT2. The load is the DC geared motor, which responds to the average of the switched voltage. The power supply block has two parts: the USB connection (or an adapter) supplies the Arduino and the 5 V logic, and an external DC supply feeds the motor through the 12V terminal of the L298N. The grounds of both are joined.

Design Considerations and Calculations

The resolution of the speed command is limited by the ADC and by the PWM generator. With a 5 V reference and a 10-bit converter, the smallest voltage step that the ADC can detect is

$$V_{step} = 5\ V / 1024 \approx 4.88\ mV \tag{4.1}$$

The potentiometer reading is reduced to 256 PWM levels, so one PWM step changes the duty cycle by

$$\Delta D = 1 / 255 \approx 0.392\ \% \tag{4.2}$$

At a PWM frequency of about 490 Hz on pin 9, the period of the waveform is

$$T = 1 / f \approx 1 / 490\ Hz \approx 2.04\ ms \tag{4.3}$$

If the external supply is V_s and the L298N has a voltage drop V_{drop} across its conducting switches (typically around 2 V for the bipolar bridge of the L298), the average motor voltage at duty cycle D is approximately

$$V_{motor} \approx D \times (V_s - V_{drop}) \tag{4.4}$$

The external supply must be chosen so that V_s is not lower than the motor's rated voltage plus V_{drop} if full rated speed is required, and its current rating must exceed the stall current of the motor. The continuous current of each L298N channel should not exceed 2 A [6].

Worked Design Example

The following example, which uses assumed values, shows how the supply and the PWM range can be chosen. Let the motor be rated at 9 V, with a running

current of 0.5 A and a stall current of 1.2 A, and let an available external supply of 12 V be used with the L298N, whose drop is taken as about 2 V.

- **Maximum voltage available at the motor:** At 100 % duty cycle the motor would receive 12 V − 2 V = 10 V, which is above its 9 V rating.
- **Limiting the duty cycle:** The maximum duty cycle that keeps the motor within its rating follows from Equation (4.4):

$$D_{max} = V_{rated} / (V_s - V_{drop}) = 9 / (12 - 2) = 0.90 \quad (4.5)$$

- **Maximum PWM value:** 0.90 × 255 ≈ 229. The program therefore maps the potentiometer range to 0–229 instead of 0–255, that is, map(potValue, 0, 1023, 0, 229).
- **Current check:** The stall current of 1.2 A is below the 2 A limit of the L298N channel, so the driver is suitable, and the supply must be able to provide at least 1.2 A.
- **Power loss in the driver:** At the running current the loss in the bridge is approximately

$$P_{loss} \approx V_{drop} \times I_m = 2\ V \times 0.5\ A = 1\ W \quad (4.6)$$

A loss of 1 W is easily handled by the heat sink on the module. If another motor or supply is used, the same steps can be repeated with its own values.

4.2 CIRCUIT DIAGRAM

Figure 4.2 shows the circuit connections of the system, and Table 4.1 lists the connections between the modules. The outer terminals of the potentiometer are connected to 5 V and GND, and the wiper is connected to the analog input A0. Pin 9, which is a PWM-capable pin, drives the ENA input of the L298N, while pins 8 and 7 drive IN1 and IN2.

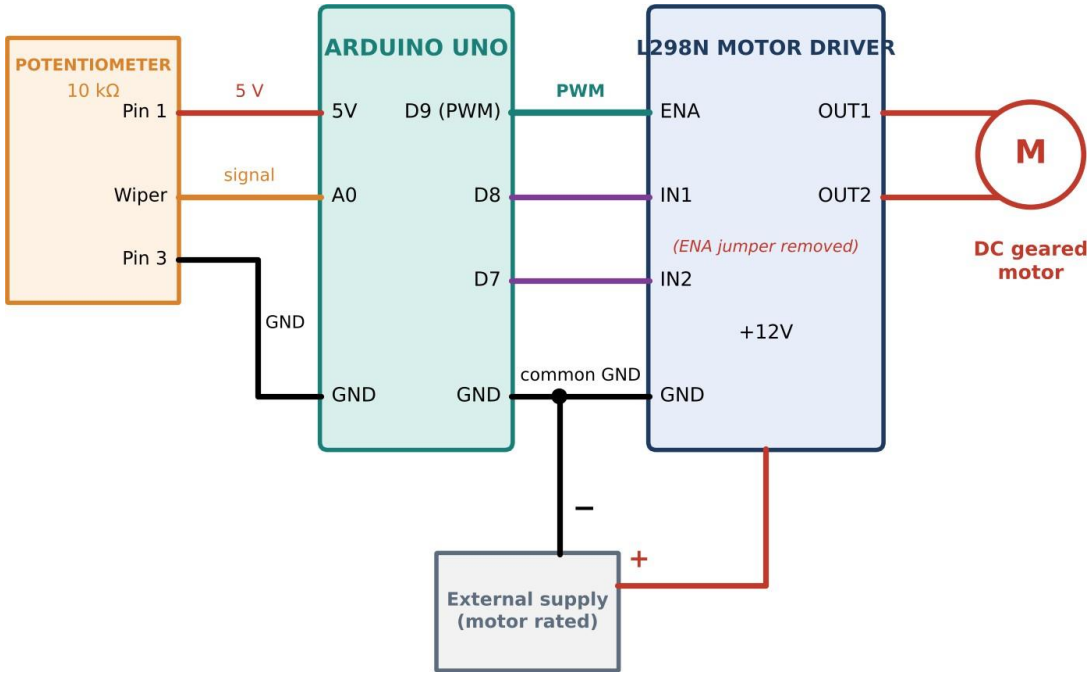


Figure 4.2 Circuit diagram of the proposed system

Table 4.1 Pin connections of the Arduino with the modules

Module	Module pin	Connected to
Potentiometer (10 kΩ)	Outer pin 1	Arduino 5V
Potentiometer (10 kΩ)	Wiper (middle pin)	Arduino A0
Potentiometer (10 kΩ)	Outer pin 3	Arduino GND
L298N module	ENA	Arduino pin 9 (PWM)
L298N module	IN1	Arduino pin 8
L298N module	IN2	Arduino pin 7
L298N module	GND	Arduino GND and external supply negative (common ground)
L298N module	12V (VS)	External supply positive
L298N module	OUT1, OUT2	Terminals of the DC geared motor

The terminals of the L298N module that appear in the circuit are summarised in Table 4.2.

Table 4.2 L298N module terminals used in the project

Terminal	Type	Function
ENA	Input	Enable input of channel A; the PWM signal from the Arduino sets the average voltage of the motor
IN1, IN2	Inputs	Direction control of channel A (see the truth table in Chapter 6)
OUT1, OUT2	Outputs	Connected to the two terminals of the DC motor
12V (VS)	Power input	Motor supply input from the external DC source
GND	Power / signal ground	Common ground of the supply, the driver and the Arduino
5V	Power	Regulated 5 V terminal of the module; the motor is never powered from it

Important Wiring Precautions

- **Remove the ENA jumper:** Most L298N modules ship with a small jumper cap across the ENA pins, which ties the enable input permanently high. With the cap fitted the motor always runs at full speed and the PWM has no effect, so the cap must be removed before the Arduino is connected to ENA.
- **Common ground:** The Arduino ground, the driver ground and the negative terminal of the motor supply must be joined. Without a common reference the logic signals are not interpreted correctly.
- **Separate motor supply:** The motor is powered from the external supply through the 12V terminal and never from the 5 V pin of the Arduino, because the pin cannot deliver the motor current.
- **On-board regulator:** The 5V-enable jumper of the module is used only when the supply voltage is within the limits stated for the module (typically up to about 12 V); at higher supply voltages it is removed and the logic is powered separately.

Layout and Noise Considerations

A motor and its driver produce electrical noise that can disturb the sensitive analog input. The wires that carry the motor current should therefore be kept short

and apart from the potentiometer and signal wires. A ceramic capacitor of about 100 nF connected directly across the motor terminals reduces the noise generated by the brushes, and an electrolytic capacitor across the motor supply terminals of the driver helps to supply the current peaks drawn at each switching edge. The potentiometer wires should be kept as short as possible and, if the reading is noisy, a small capacitor can be placed between the wiper and ground. These measures are inexpensive and improve the stability of the reading and the reliability of the system.

4.3 FLOWCHART

Figure 4.3 shows the program flow of the controller. The program has a short initialisation part, which runs once, followed by a loop that repeats indefinitely.

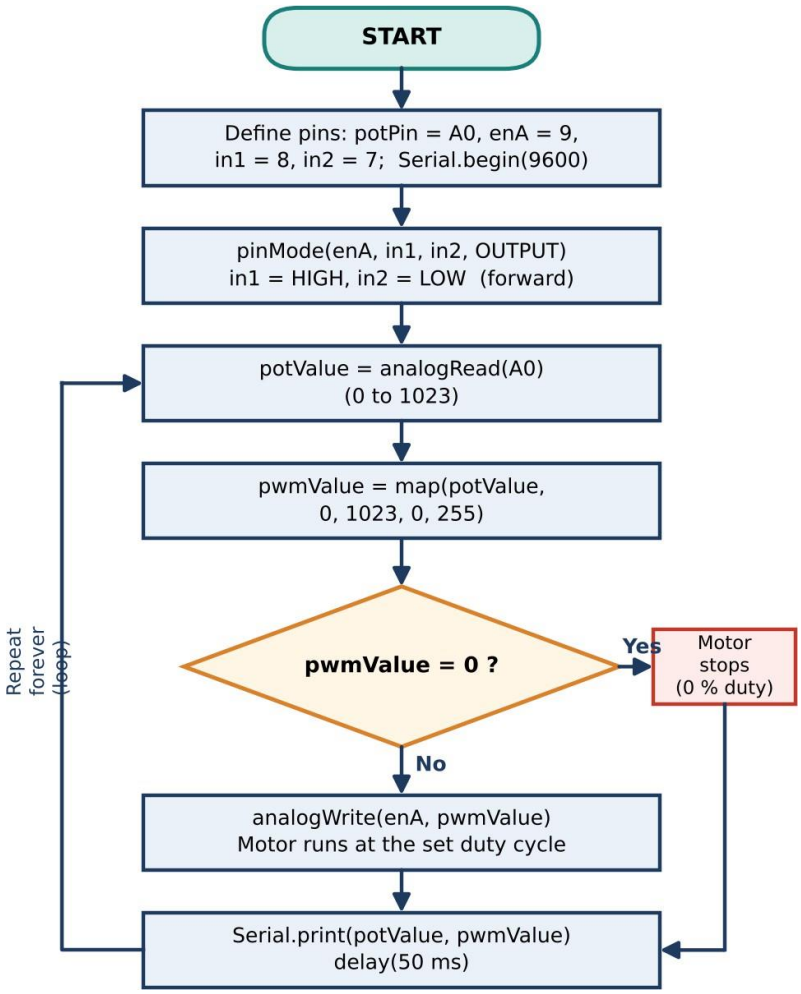


Figure 4.3 Flowchart of the system operation

After the program starts, the pin numbers are defined and the serial port is opened at 9600 baud. The three control pins are configured as outputs, and the direction pins are set to IN1 = HIGH and IN2 = LOW so that the motor rotates in the forward direction. The main loop then reads the potentiometer with analogRead(), converts the value to 0–255 with map(), and writes it to pin 9 with analogWrite().

When the PWM value is zero, the output stays low and the motor stops. For any other value the PWM signal drives the ENA input and the motor runs at the corresponding speed. The potentiometer value and the PWM value are then printed on the serial monitor, and a delay of 50 ms keeps the loop from running

faster than necessary and makes the serial output readable. The loop then returns to read the potentiometer again, so the speed follows the knob continuously.

Algorithm

- Step 1: Start the program and define the pins potPin (A0), enA (9), in1 (8) and in2 (7).
- Step 2: Configure enA, in1 and in2 as outputs, set in1 = HIGH and in2 = LOW, and start the serial port at 9600 baud.
- Step 3: Read the potentiometer: potValue = analogRead(A0).
- Step 4: Convert the reading: pwmValue = map(potValue, 0, 1023, 0, 255).
- Step 5: Write the PWM value to the enable pin: analogWrite(enA, pwmValue).
- Step 6: Print potValue and pwmValue on the serial monitor and wait for 50 ms.
- Step 7: Go to Step 3 and repeat for as long as power is applied.

CHAPTER 5
IMPLEMENTATION

This chapter details the physical construction of the speed controller prototype and the program that runs on the Arduino. The system was realised by wiring the potentiometer and the L298N to the Arduino as in the circuit diagram and by writing a short C++ program that converts the knob position into a PWM signal.

5.1 HARDWARE IMPLEMENTATION

The prototype was assembled in a modular way, with the low-power control circuit on a breadboard and the motor wiring kept separate on the screw terminals of the L298N module. The following steps describe the assembly.

- **Potentiometer wiring:** The potentiometer is placed on the breadboard. One outer pin is connected to the 5V pin of the Arduino, the other outer pin to GND, and the middle pin (wiper) to the analog input A0. The orientation of the outer pins decides only whether the speed increases clockwise or anticlockwise.
- **Driver connections:** The jumper cap on the ENA terminal of the L298N is removed. Pin 9 of the Arduino is wired to ENA, pin 8 to IN1 and pin 7 to IN2. The GND terminal of the L298N is wired to a GND pin of the Arduino.
- **Motor connection:** The two wires of the DC geared motor are fixed in the OUT1 and OUT2 screw terminals. If the shaft turns in the opposite direction to the expected one, the two motor wires can be swapped or the logic levels of IN1 and IN2 can be exchanged in the program.
- **Power connection:** The positive wire of the external supply is fixed in the 12V terminal and its negative wire in the GND terminal of the L298N, so that the supply, the driver and the Arduino share one ground. The supply is switched on only after all the connections have been checked.

Pre-power Checklist

Before applying the motor supply the following points were checked: the ENA jumper is removed; there is no short between 5 V and GND on the breadboard; the polarity of the supply wires is correct; the supply voltage does not exceed the rating of the motor; and the motor shaft is free to rotate. The Arduino can be programmed and the potentiometer tested through the serial monitor first, with the motor supply still switched off. This confirms that the A0 readings change from about 0 to 1023 over the full travel of the knob.

Safety Precautions

- Never connect the motor supply to the 5 V pin or to any signal pin of the Arduino.
- Switch off the external supply before changing any connection on the L298N screw terminals.
- Keep the supply voltage within the limits of the module and of the motor, and do not leave a stalled motor energised, since the current and the heating of the driver rise quickly.
- The heat sink of the L298N can become hot during use. It should not be touched while the system is running at a high load.
- Keep loose wires, tools and fingers away from the rotating shaft and gears.

Sequence of Hardware Testing

The hardware is tested in stages so that any fault can be located easily. First the Arduino is tested alone with the potentiometer by observing the serial monitor. Next the L298N is connected with the motor supply switched off, and the wiring is checked against Table 4.1. The motor supply is then switched on at a low voltage with the knob at its minimum, and the knob is advanced slowly while the motor and the driver are observed. Only after all stages work correctly is the final supply voltage applied.

5.2 SOFTWARE IMPLEMENTATION

The program was written in the Arduino IDE (v2.x) in C++ and uploaded to the board through the USB cable. The structure follows the flowchart of Figure 4.3, and the program uses only functions of the Arduino core.

1. Pin Definitions

Constants give meaningful names to the pins: potPin for A0, enA for pin 9, and in1 and in2 for pins 8 and 7. Using constants rather than numbers inside the code makes the program easier to read and to change if the wiring is modified. Pin 9 is used for ENA because it is one of the six pins that support PWM on the UNO.

2. Reading the Potentiometer

In every pass of the loop, analogRead(potPin) returns an integer between 0 and 1023 that is proportional to the voltage at the wiper. When the knob is turned fully towards the ground end the value is close to 0, and when it is turned fully towards the 5 V end the value is close to 1023.

3. Mapping to a PWM Value

The PWM function accepts only values from 0 to 255, so the reading is scaled with map(potValue, 0, 1023, 0, 255). Because map() uses integer arithmetic, the result is truncated, so a reading of 512 gives 127 and not 128. Figure 5.1 shows the linear relationship, and Table 5.1 lists the PWM value, the duty cycle and the potentiometer voltage for selected readings.

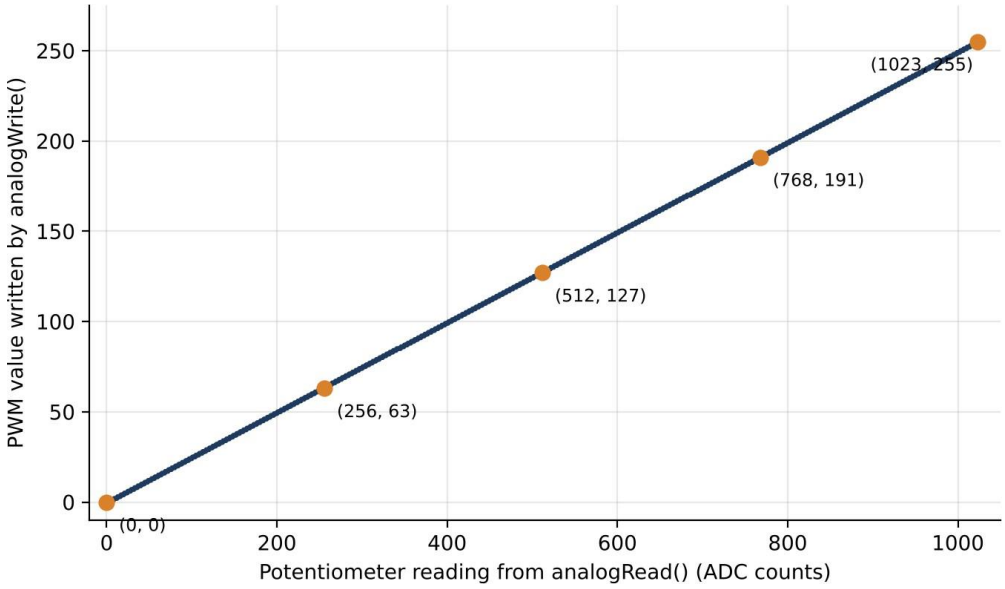


Figure 5.1 Mapping of the potentiometer reading to the PWM value

Table 5.1 Mapping of potentiometer readings to PWM values

ADC reading (0–1023)	Wiper voltage (V)	PWM value (0–255)	Duty cycle (%)
0	0.00	0	0.0
128	0.63	31	12.2
256	1.25	63	24.7
384	1.88	95	37.3
512	2.50	127	49.8
640	3.13	159	62.4
768	3.75	191	74.9
896	4.38	223	87.5
1023	5.00	255	100.0

4. Generating the PWM Signal

The call analogWrite(enA, pwmValue) loads the compare register of the hardware timer that serves pin 9. The timer then toggles the pin without any further effort from the processor, so the program can continue to read the potentiometer while the PWM runs in the background. Pin 9 is driven by Timer1 of the ATmega328P, which gives a PWM frequency of approximately 490 Hz [7], [8].

5. Serial Monitoring

The values of potValue and pwmValue are sent to the computer through Serial.print() at 9600 baud. This helps in checking the wiring, in observing the dead zone of the motor and in recording the readings for the results table. A delay of 50 ms between passes of the loop limits the output to about 20 lines per second.

6. Complete Program

The complete program of the speed controller is given below.

```
const int potPin = A0;   // potentiometer wiper input
const int enA    = 9;    // PWM output to L298N ENA
const int in1    = 8;    // direction input 1
const int in2    = 7;    // direction input 2

void setup() {
  pinMode(enA, OUTPUT);
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);

  // Fixed direction: forward
  digitalWrite(in1, HIGH);
  digitalWrite(in2, LOW);

  Serial.begin(9600);
}

void loop() {
  int potValue = analogRead(potPin);               // 0 to 1023
  int pwmValue = map(potValue, 0, 1023, 0, 255);   // 0 to 255

  analogWrite(enA, pwmValue);                      // set duty cycle

  Serial.print("Pot: ");
  Serial.print(potValue);
  Serial.print("  PWM: ");
  Serial.println(pwmValue);

  delay(50);
}
```

7. Optional Improvements to the Program

Two small changes make the controller behave better with real motors. First, the potentiometer reading can be averaged over several samples to remove the flicker caused by electrical noise. Second, because a geared motor usually does not start below a certain duty cycle, the PWM range can be started at a minimum value, minPwm, found by testing, and a small dead band at the bottom

of the knob can be used to switch the motor off. The following version applies both changes; the value of minPwm must be determined for the motor in use.

```
const int potPin = A0;
const int enA = 9, in1 = 8, in2 = 7;
const int minPwm = 60;        // set after testing the motor
const int deadBand = 20;      // pot counts treated as "off"

int readSmooth() {
  long sum = 0;
  for (int i = 0; i < 8; i++) {
    sum += analogRead(potPin);
  }
  return sum / 8;             // average of 8 samples
}

void setup() {
  pinMode(enA, OUTPUT);
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);
  digitalWrite(in1, HIGH);
  digitalWrite(in2, LOW);
  Serial.begin(9600);
}

void loop() {
  int potValue = readSmooth();
  int pwmValue;

  if (potValue < deadBand) {
    pwmValue = 0;             // motor off
  } else {
    pwmValue = map(potValue, deadBand, 1023, minPwm, 255);
  }

  analogWrite(enA, pwmValue);
  Serial.print(potValue);
  Serial.print(",");
  Serial.println(pwmValue);
  delay(50);
}
```

8. Calibrating the Starting Threshold

The value of minPwm used in the second program depends on the motor, the load and the supply voltage, so it is found by experiment. The procedure is as follows:

- Upload the first program and open the serial monitor. Set the knob to its minimum position.
- Turn the knob very slowly until the shaft just begins to turn without help. Note the PWM value shown on the monitor at that moment.

- Repeat the test a few times from different starting positions and take a value slightly above the highest reading, so that the motor starts reliably.
- Enter this number as minPwm in the second program, and check that the motor now starts smoothly when the knob is moved just beyond the dead band.

9. Common Problems and Remedies

Table 5.2 lists the problems that are most often met when the circuit is built for the first time, together with their probable causes and remedies.

Table 5.2 Common problems and remedies

Symptom	Probable cause	Remedy
Motor runs at full speed whatever the knob position	ENA jumper still fitted on the L298N	Remove the jumper cap and connect pin 9 to ENA
Motor does not run at all	No motor supply, or ground not common, or ENA not connected	Check the supply, join the grounds and verify the ENA wire
Motor hums but does not turn at low settings	Duty cycle below the starting threshold (dead zone)	Increase the setting or start the PWM range at minPwm
Direction is opposite to the expected one	Motor wires or IN1/IN2 levels exchanged	Swap the motor wires or the levels of IN1 and IN2
Reading on the serial monitor jumps or flickers	Noise on the wiper or a loose potentiometer connection	Use shorter wires, average several samples, add a small capacitor at A0
Arduino resets when the motor starts	Motor supply drawn from a weak source or noise on the 5 V line	Use a separate, stronger supply and add capacitors across the supply
L298N becomes very hot	High current or large voltage drop at low supply voltage	Reduce the load, use a heat sink or choose a more efficient driver

CHAPTER 6
WORKING OF THE SYSTEM

This chapter explains the operation of the DC motor speed controller step by step. The system works as a continuous loop: the user sets the knob, the Arduino converts the position into a PWM signal, the driver switches the motor supply at that duty cycle, and the motor settles at the corresponding speed.

1. System Initialisation

When power is applied, the Arduino boots and runs setup() once. The three control pins are configured as outputs and the serial port is opened. The direction pins are then set to IN1 = HIGH and IN2 = LOW, which selects forward rotation for the whole time the program is running. At this moment the PWM output is not yet written, so the ENA pin is low and the motor is at rest.

2. Reading the Speed Command

The loop() function starts by reading the potentiometer. The wiper voltage, which lies between 0 V and 5 V, is applied to the ADC through pin A0. The ADC compares it with the 5 V reference and returns a 10-bit number. For example, with the knob at the centre the wiper voltage is about 2.5 V and the reading is about 512.

3. Scaling to a PWM Value

The program converts the 10-bit reading to an 8-bit value with map(). A reading of 512 becomes 127, and a reading of 1023 becomes 255. This number is the speed command in the form needed by the PWM generator.

4. PWM Generation

The call to analogWrite() sets the duty cycle of the timer for pin 9 to the PWM value divided by 255. The pin then switches between 0 V and 5 V at about 490 Hz, staying high for the fraction of each period given by the duty cycle. For a PWM value of 127 the pin is high for about half of every period, and for a value of 255 it stays high continuously.

5. Switching of the Motor Supply by the L298N

The PWM signal at ENA enables and disables the bridge. While ENA is high and IN1 = HIGH, IN2 = LOW, the switches S1 and S4 of the H-bridge conduct, and the current of the external supply flows from OUT1 through the motor to OUT2. While ENA is low, the bridge is switched off and the motor is no longer driven. Table 6.1 shows the logic states of the module.

Table 6.1 Logic states of the L298N for one channel

ENA	IN1	IN2	Motor action
LOW	X	X	Motor not driven (free running stop)
PWM / HIGH	HIGH	LOW	Forward rotation, speed set by the duty cycle
PWM / HIGH	LOW	HIGH	Reverse rotation, speed set by the duty cycle
HIGH	HIGH	HIGH	Fast motor stop (brake)
HIGH	LOW	LOW	Fast motor stop (brake)

6. Response of the Motor

The armature of a DC motor can be described by the voltage equation

$$V = I_a R_a + L_a (dI_a / dt) + K_e \omega \tag{6.1}$$

where V is the applied voltage, I_a the armature current, R_a and L_a the armature resistance and inductance, K_e the back-EMF constant and ω the angular speed. The developed torque is

$$T = K_t I_a \tag{6.2}$$

For the switching frequency used here, the inductance L_a opposes rapid changes in the current, and the inertia of the rotor prevents rapid changes in the speed. The motor therefore responds to the average voltage, and in the steady state the speed is approximately

$$\omega \approx (V_{avg} - I_a R_a) / K_e \tag{6.3}$$

Equation (6.3) shows why the speed is almost proportional to the duty cycle. A load that needs more torque draws more current, increases the $I_a R_a$ drop and lowers the speed slightly for the same duty cycle, which is the main limitation

of open-loop control. When the bridge switches off, the motor current continues to flow through the flyback diodes of the module, which prevents high voltage spikes that could damage the driver.

7. Speed Variation with the Knob Position

- **Knob at minimum:** The reading is near 0 and the PWM value is 0, so ENA stays low and the motor is stopped.
- **Knob at an intermediate position:** The PWM value is between 0 and 255, the duty cycle is partial, and the average motor voltage and speed are proportional to the setting. Below a certain duty cycle the motor may hum without turning, because the torque is not enough to overcome static friction and the gear train.
- **Knob at maximum:** The PWM value is 255 and ENA stays high continuously. The motor receives the full supply voltage less the drop of the L298N and runs at its highest speed.

Figure 6.1 summarises the signal chain for three positions of the knob, and Table 6.2 lists the corresponding values at each stage. The motor voltages are calculated for a 12 V supply with a driver drop of about 2 V, as in Chapter 4.

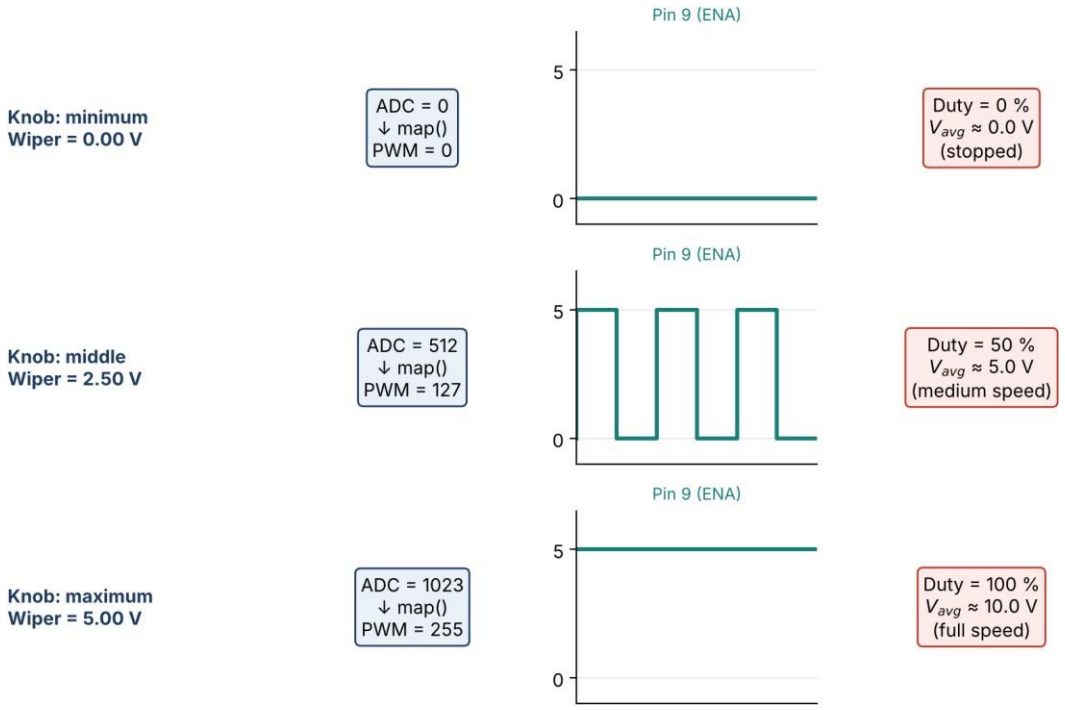


Figure 6.1 Signal chain from the knob position to the motor voltage

Table 6.2 Signal values at each stage for three knob settings

Stage	Knob at minimum	Knob at middle	Knob at maximum
Wiper voltage at A0 (V)	0.00	2.50	5.00
ADC reading (0–1023)	0	512	1023
PWM value (0–255)	0	127	255
Duty cycle (%)	0	49.8	100
Average motor voltage, calculated (V)	0.00	4.98	10.00
Motor state	Stopped	Medium speed	Full speed

8. Continuous Monitoring

After the PWM value is written, the program prints the potentiometer and PWM values on the serial monitor and waits for 50 ms. The loop then restarts and reads the potentiometer again. Since the loop repeats about twenty times a second, any change of the knob is reflected in the motor speed almost immediately.

9. Starting, Stopping and Changing the Speed

When the knob is moved away from the minimum position, the duty cycle rises and the average voltage increases, but the motor does not reach its new speed

instantly. The rotor inertia and the inductance of the winding give the speed a smooth rise, and a similar smooth fall occurs when the knob is turned back. This behaviour acts as a natural soft start and avoids a sudden jerk in the gears. When the PWM value becomes zero the enable input stays low, the bridge is switched off and the motor coasts to rest, with the stopping time depending on the inertia and friction of the load. The fast-stop (brake) states of Table 6.1 are not used in this program.

10. Worked Example

Consider the knob at a position where analogRead() returns 640. The wiper voltage is

$$V_{wiper} = 640 \times 5\ V / 1023 \approx 3.13\ V \tag{6.4}$$

The program computes the PWM value as 640 × 255 / 1023 = 159 (after truncation), so the duty cycle is

$$D = 159 / 255 \approx 62.4\ \% \tag{6.5}$$

For a 12 V supply and a driver drop of about 2 V, Equation (4.4) gives the average motor voltage

$$V_{motor} \approx 0.624 \times (12 - 2) \approx 6.24\ V \tag{6.6}$$

The motor therefore runs at a little more than 60 % of the speed it would reach at full duty cycle, if the load and the friction effects are neglected.

CHAPTER 7
RESULTS AND DISCUSSION

7.1 EXPERIMENTAL SETUP

The experiment was carried out on the breadboard prototype described in Chapter 5. The Arduino was connected to a computer through the USB cable, so that the serial monitor could display the potentiometer and PWM values. The DC geared motor was connected to the OUT1 and OUT2 terminals of the L298N and powered from an external DC supply. A digital multimeter was used to measure the voltage at the motor terminals, and a tachometer, or a simple mark on the shaft counted over a fixed time, can be used to measure the speed of the output shaft.

Test Procedure

- Upload the program of Section 5.2 to the Arduino and open the serial monitor at 9600 baud.
- With the motor supply switched off, turn the potentiometer from one end to the other and confirm that the displayed values change from about 0 to about 1023 and from 0 to 255.
- Switch on the external supply and set the knob to the minimum position. The motor should be at rest.
- Increase the knob in steps (for example in steps of about 128 counts). At each step wait until the speed settles, then note the potentiometer value, the PWM value, the average voltage across the motor terminals and the shaft speed.
- Repeat the readings while decreasing the knob position, and note the lowest PWM value at which the motor just starts turning (the starting threshold) and the value at which it stops.

7.2 OBSERVED RESULTS

Table 7.1 gives the expected values of the PWM duty cycle and of the average motor voltage for nine positions of the knob. The PWM value is

calculated with the same integer arithmetic as the program. The motor voltage is calculated from Equation (4.4) for a 12 V supply and an assumed L298N drop of about 2 V, so it must be replaced by the measured voltage if the supply or the motor is different. The shaded columns are to be filled in with the readings taken from the prototype.

Table 7.1 PWM values, duty cycle and motor voltage for different knob positions

ADC reading	PWM value	Duty cycle (%)	Calculated V_{avg} (V)	Measured V_{avg} (V)	Measured speed (RPM)
0	0	0.0	0.00		
128	31	12.2	1.22		
256	63	24.7	2.47		
384	95	37.3	3.73		
512	127	49.8	4.98		
640	159	62.4	6.24		
768	191	74.9	7.49		
896	223	87.5	8.75		
1023	255	100.0	10.00		

The relation between the knob position and the duty cycle is a straight line, as plotted in Figure 7.1. The shaded region at the bottom marks the range of duty cycles in which many geared motors fail to start because of static friction and the gear train. The exact limit depends on the motor and on the load, so it must be found by experiment.

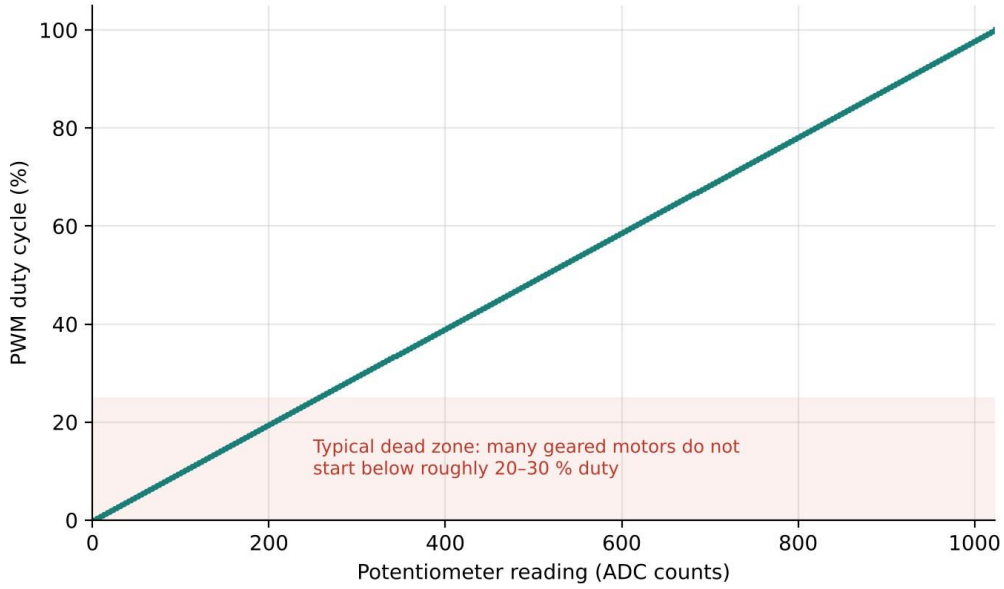


Figure 7.1 PWM duty cycle versus potentiometer reading

Figure 7.2 shows the average motor voltage as a function of the duty cycle. The dashed line is the ideal relation of Equation (1.2) for a 12 V supply, and the solid line includes the voltage drop of about 2 V across the L298N. The drop reduces the maximum voltage available to the motor, and the loss in the driver appears as heat.

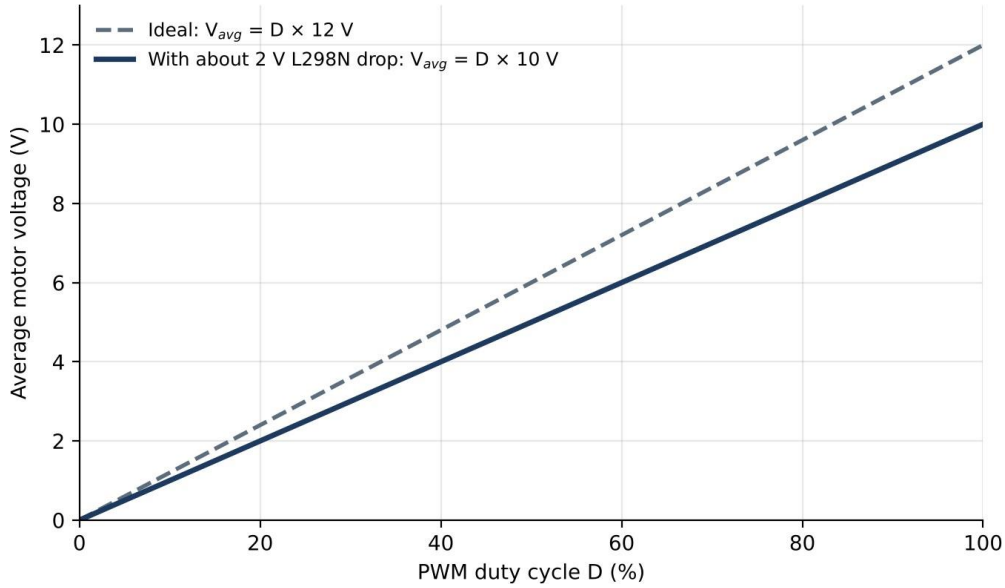


Figure 7.2 Calculated average motor voltage versus duty cycle

Table 7.2 lists the functional tests that were performed on the prototype. The last column is left for the observation recorded for each test.

Table 7.2 Functional tests of the speed controller

S.No	Test condition	Expected behaviour	Observation
1	Knob at minimum	PWM = 0; motor stationary	
2	Knob at about 25 %	Low duty cycle; motor may hum or turn very slowly (dead zone)	
3	Knob at mid position	Duty cycle about 50 %; medium speed	
4	Knob at maximum	PWM = 255; full speed, motor voltage = supply less driver drop	
5	Knob turned slowly through the range	Speed rises smoothly without jumps	
6	ENA jumper left on the module (negative test)	Motor runs at full speed whatever the knob position; PWM has no effect	
7	IN1 and IN2 exchanged in the program	Direction of rotation reverses; speed control unchanged	

The supply voltage has a direct influence on the voltage available to the motor. Table 7.3 gives the calculated average motor voltage for three supply voltages and four duty cycles, using Equation (4.4) with a driver drop of about 2 V. The values are plotted in Figure 7.3.

Table 7.3 Calculated average motor voltage for different supply voltages

Duty cycle (%)	V_s = 6 V	V_s = 9 V	V_s = 12 V
25	1.00 V	1.75 V	2.50 V
50	2.00 V	3.50 V	5.00 V
75	3.00 V	5.25 V	7.50 V
100	4.00 V	7.00 V	10.00 V

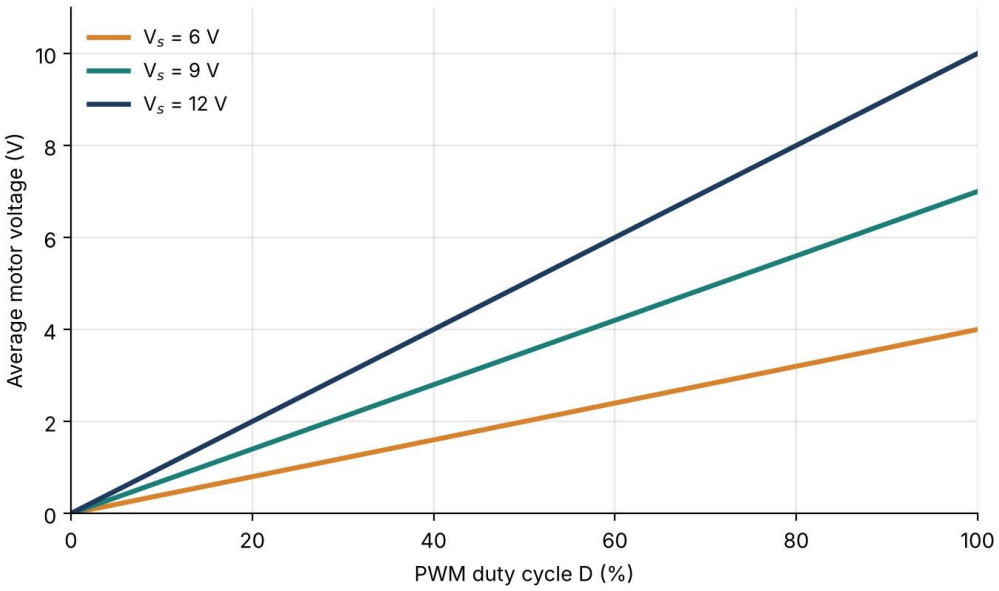


Figure 7.3 Calculated average motor voltage for different supply voltages

The figure shows that a higher supply voltage extends the speed range of the motor, and that at a low supply voltage the fixed drop of the driver takes a large share of the voltage. A 6 V supply, for example, gives only 4 V at the motor at full duty cycle, which would not be enough for a 6 V motor to reach its rated speed.

7.3 DISCUSSION

1. Linearity

The mapping from the knob position to the PWM value is linear, and Equations (1.2) and (6.3) predict that the motor speed is roughly proportional to the duty cycle above the starting threshold. A graph of the measured speed against the duty cycle should therefore be close to a straight line, except at low duty cycles where friction dominates and at high loads where the voltage drop in the armature resistance becomes important.

2. Dead Zone at Low Duty Cycles

A geared motor needs a minimum voltage to produce enough torque to overcome static friction in the motor and in the gear train, and the L298N also takes about 2 V from the supply. As a result, the motor typically does not rotate

over the first part of the knob travel, even though the PWM value is already non-zero. The program of Section 5.2 (point 7) compensates for this by starting the PWM range at minPwm, so that the whole knob travel becomes useful.

3. Resolution and Stability of the Speed Setting

The ADC gives 1024 levels but the PWM has only 256, so about four neighbouring ADC readings produce the same PWM value, and the speed changes in steps of 0.39 % of the supply. This is much finer than a person can perceive on a rotating shaft. Small electrical noise on the wiper can, however, make the last digit of the reading flicker, and averaging a few samples removes this effect.

4. PWM Frequency

The 490 Hz PWM frequency of pin 9 is in the audible range, and some motors produce a faint whine at low duty cycles. Raising the frequency above 20 kHz by changing the timer prescaler removes the noise, but the switching losses in the driver increase and the L298N, which uses slow bipolar transistors, becomes less efficient. For the small motor of this prototype the default frequency is a reasonable compromise.

5. Efficiency and Heating of the Driver

The L298N loses about 2 V across its switches at the rated current, so a driver supplied with 12 V and delivering 1 A dissipates around 2 W, which is why the module has a heat sink. At low duty cycles the motor current falls and the loss is smaller. The PWM method itself does not waste power in a series element, which is the main advantage over resistor or linear control.

6. Open-Loop Operation

Because the controller sets only the average voltage, the speed changes when the load, the supply voltage or the temperature changes. If the motor is loaded it slows down for the same knob position. Where an accurate speed is

needed, a speed sensor and a feedback loop must be added, as discussed in Section 9.2.

7. Effect of the Supply Voltage

Because the PWM duty cycle is a fraction of the supply voltage, any change in the supply, such as the fall in the voltage of a battery during use, changes the motor speed at the same knob position. A regulated supply keeps the speed repeatable, while a battery-powered system benefits from feedback.

8. Effect of the Load

A mechanical load increases the armature current and, according to Equation (6.3), lowers the speed for a fixed average voltage. In a geared motor the load seen by the motor is reduced by the gear ratio, so the effect is smaller than for a direct-drive motor, but it can still be noticed when the output shaft is held by hand. This is the reason for the speed feedback proposed in the future scope.

9. Sources of Error

- **ADC quantisation and noise:** The reading can vary by one or two counts, which changes the PWM value by a fraction of a step.
- **Supply and driver drop:** The drop across the L298N depends on the current, so the voltage at the motor is not exactly $D \times (V_s - 2\ V)$.
- **Measurement of a pulsed voltage:** A multimeter that is not a true-RMS or averaging type may show a value that differs from the average voltage of the PWM waveform, so the readings should be taken with care.
- **Speed measurement:** Counting revolutions by eye or with a simple tachometer limits the accuracy of the speed readings.

10. Comparison with the Reviewed Work

The behaviour of the prototype agrees in principle with the earlier work [1], [3], [4]: the speed varies with the PWM duty cycle. The present controller

keeps the circuit to a minimum number of parts and documents the mapping from the knob to the motor voltage in Tables 5.1 and 7.1.

CHAPTER 8

ADVANTAGES, LIMITATIONS AND APPLICATIONS

8.1 ADVANTAGES

- **Simple and Low Cost:** The system needs only an Arduino, an L298N module, a potentiometer and a motor, all of which are inexpensive and easy to obtain.
- **Efficient Speed Control:** PWM varies the average voltage by switching, so no power is wasted in a series resistor or a linear regulator and the motor receives almost the full supply voltage at maximum setting.
- **Smooth and Continuous Adjustment:** The speed can be varied from zero to maximum by turning the knob, with a resolution of 256 levels.
- **Safe Interface:** The motor current flows through the driver and the external supply, and the Arduino pins carry only logic signals, so the controller is protected from the motor.
- **Easy to Understand and Modify:** The program is short and uses only standard Arduino functions, and the system can easily be extended with direction control, sensors or feedback.
- **Easy Monitoring:** The potentiometer and PWM values are available on the serial monitor, which makes testing and calibration straightforward.
- **Low Power for the Control Circuit:** The Arduino and the potentiometer draw very little power, so almost all the energy goes to the motor, and the controller can also be run from a small battery or a USB supply.
- **Expandable Design:** Extra inputs, sensors, displays and communication modules can be added to the same board without changing the basic circuit, which makes the project a good starting point for more advanced work.

8.2 LIMITATIONS

- **Open-Loop Control:** The actual speed is not measured, so it changes with the load and with the supply voltage.

- **Voltage Drop and Heating in the L298N:** The bipolar bridge drops about 2 V and dissipates heat, so the driver is inefficient at low supply voltages and for high currents.
- **Dead Zone at Low Speed:** The motor does not start below a certain duty cycle, so the lowest part of the knob travel has no effect unless the program compensates for it.
- **PWM Noise:** The default 490 Hz frequency may cause an audible whine in some motors.
- **Current Limit:** Each channel of the L298N is limited to about 2 A, so the module cannot drive large motors.
- **Fixed Direction:** In this version the direction is fixed in the program and cannot be changed from the user interface.
- **No Protection Features:** The prototype has no current limit, over-temperature indication or fuse of its own, so the user must take care not to stall the motor or overload the driver.
- **Dependence on Mechanical Condition:** The speed at a given setting changes as the gears and brushes wear and as the lubrication changes, because no measurement of the real speed is made.

8.3 APPLICATIONS

- **Robotics:** Controlling the speed of the wheels of small robotic vehicles and the joints of simple arms.
- **Fans and Blowers:** Variable-speed cooling fans, ventilation fans and small blowers.
- **Pumps:** Adjusting the flow of small DC water or liquid pumps.
- **Conveyor Prototypes:** Laboratory models of conveyor belts and material handling systems.
- **Educational Laboratories:** Teaching PWM, motor drives, analog-to-digital conversion and embedded programming.

- **Home and Hobby Projects:** Motorised curtains, turntables, model trains and similar variable-speed devices.
- **Automatic Doors and Shutters:** Controlling the opening speed of small doors, shutters and sliding mechanisms.
- **Model Vehicles and Toys:** Speed control of remote-controlled cars, boats and trains.

CHAPTER 9
CONCLUSION AND FUTURE SCOPE

9.1 CONCLUSION

The DC Motor Speed Control Using Arduino and PWM project showed that the speed of a DC geared motor can be controlled smoothly and efficiently with a very small number of components. A potentiometer acts as the speed-setting device, the Arduino reads it with its 10-bit ADC and converts the value into an 8-bit PWM signal, and the L298N driver switches the motor supply at the corresponding duty cycle.

The relation between the knob position, the PWM value, the duty cycle and the average motor voltage was worked out, tabulated and plotted, and the practical requirements of the design were identified: removal of the ENA jumper, a common ground, a separate motor supply and the allowance for the voltage drop of the L298N and for the dead zone of the motor at low duty cycles.

The system is simple, low cost and easy to build and to modify. Its open-loop nature and the losses of the L298N are the main limitations, but they can be overcome with feedback and a more efficient driver, as suggested in the future scope. The project therefore provides a sound basis for more advanced motor-control systems and for teaching the principles of PWM and microcontroller-based drives.

Besides the technical outcome, the project gave practical experience in reading datasheets, wiring a mixed analog and power circuit, writing and debugging an embedded program, and recording and presenting measurements. These skills are directly useful in the design of larger embedded and drive systems, and the report can serve as a reference for others who wish to repeat or extend the work.

9.2 FUTURE SCOPE

The prototype demonstrates the basic principle, and a number of enhancements can be added to make the system more accurate and more capable:

- **Closed-Loop Speed Control:** An encoder or an IR tachometer can measure the actual speed, and a PID algorithm can adjust the duty cycle to hold the set speed even when the load or the supply changes.
- **Bidirectional Control:** A switch or a second input can be added so that the direction of rotation can be changed from the user interface, with the PWM value reduced to zero before reversal.
- **More Efficient Driver:** The L298N can be replaced by a MOSFET-based driver, which has a much smaller voltage drop, runs cooler and can supply higher currents.
- **Higher PWM Frequency:** The timer prescaler can be changed to raise the PWM frequency above the audible range (about 31 kHz on pin 9), which removes motor whine.
- **Display of Speed:** An LCD or OLED display can show the set speed, the duty cycle and the measured RPM.
- **Current Sensing and Protection:** A current sensor can detect a stalled motor or an over-current condition and switch the drive off.
- **Wireless and IoT Control:** The knob can be replaced or supplemented by a Bluetooth or Wi-Fi link so that the speed can be set from a smartphone or logged to a cloud dashboard.

With these additions, the simple speed controller of this project can grow into a complete motor-control platform that is suitable for robotics, automation and industrial prototypes, while keeping the same basic principle of using PWM to set the average motor voltage.

REFERENCES

[1] N. P. Joshi and A. P. Thakare, “Speed control of DC motor using analog PWM technique,” International Journal of Engineering Research & Technology (IJERT), vol. 1, no. 9, 2012.

[2] M. S. Islam and V. K. Tripathi, “A study of D.C. motor speed control through pulse width modulation implemented by MATLAB simulation,” International Journal of Advanced Research in Computer and Communication Engineering (IJARCCE), vol. 5, no. 6, Jun. 2016.

[3] D. B. Mane, R. B. Koli, S. S. Bhosale and V. V. Jogdand, “Arduino based speed control of DC motor by using LabVIEW,” International Journal of Innovative Research in Science, Engineering and Technology (IJIRSET), vol. 6, no. 5, May 2017.

[4] S. Banerjee, S. Mondal, B. Saha and A. J. Mandal, “Speed control of a DC motor using pulse width modulation, potentiometer and IR sensor,” International Journal of Advanced Research in Science, Engineering and Technology (IJARSET), vol. 5, no. 8, Aug. 2018.

[5] T. Özer, S. Kıvrak and Y. Oğuz, “H bridge DC motor driver design and implementation with using dsPIC30F4011,” International Journal of Innovative Research in Science, Engineering and Technology (IJIRSET), vol. 6, Special Issue 10, 2017.

[6] STMicroelectronics, “L298: Dual full-bridge driver,” datasheet.

[7] Microchip Technology Inc., “ATmega328P: 8-bit AVR microcontroller with 32 KB in-system programmable flash,” datasheet.

[8] Arduino, “analogWrite(),” Arduino Language Reference, docs.arduino.cc.

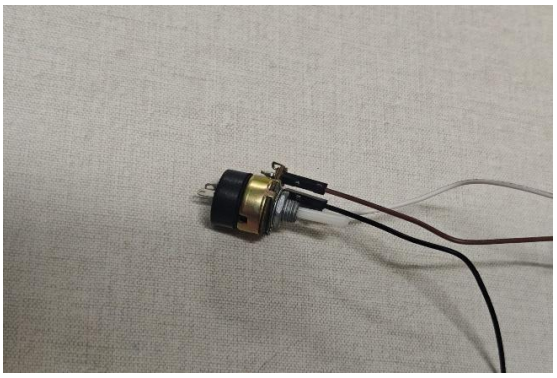
[9] M. H. Rashid, Power Electronics: Circuits, Devices and Applications, Pearson Education.

[10] S. J. Chapman, Electric Machinery Fundamentals, McGraw-Hill Education.

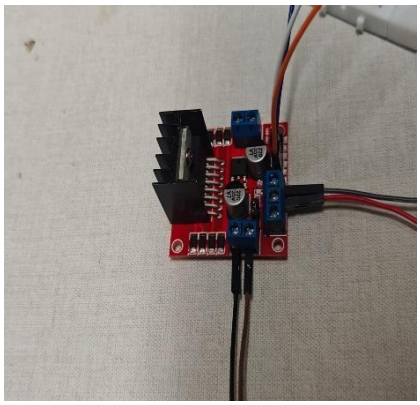
OUTCOMES OF PROJECT



Motor



Potentiometer



Motor Driver (L298N)



Arudino Uno

